# Pre-Lab Methodology Report:
# Phase-Conditioned Structured Degradation Reward Learning from Suboptimal Demonstrations

Jaemin Cho

June 2026

**Abstract**

Learning from Demonstrations (LfD) allows robots to acquire manipulation skills from observed behavior. However, demonstrations are often not optimal. A demonstration may successfully complete the task while still containing inefficient motion patterns such as sidetracks, unnecessary pauses, repeated alignment corrections, high jerk, unstable contact, or overly long paths. This pre-lab report proposes a methodology for learning better-than-demonstrator (BfD) policies from successful-but-inefficient demonstrations.

The proposed framework follows the spirit of SSRR/D-REX-style ranking-based reward learning. Instead of injecting random Gaussian noise, the method uses goal- and phase-conditioned structured action perturbations to generate a weak ordinal degradation hierarchy, such as $\tau_{\text{orig}} \succ \tau_{\text{mild}} \succ \tau_{\text{med}} \succ \tau_{\text{sev}}$. A reward network is trained by pairwise preference fitting over phase-conditioned consequence features. The central design choice is that the reward should not learn simple similarity to the original demonstration. Instead, it should learn the latent degradation direction induced by structured perturbations. Policy improvement then searches for trajectories with lower predicted inefficiency than the original demonstration. Since this is a pre-lab proposal, the reward extrapolation required for BfD is treated as an empirical hypothesis to be tested in simulation.

## 1 Introduction

### 1.1 Motivation

Learning from Demonstrations is attractive for robot manipulation because manually designing rewards for complex tasks is difficult. However, demonstrations are rarely optimal. Even a successful demonstration may include unnecessary or inefficient execution patterns. For example, in a pick-and-place task, a robot may move the object to the target while still showing unnecessary sidetrack motion, repeated pre-grasp alignment, excessive jerk during transport, unstable contact, unnecessary pauses, or longer-than-needed paths.

Standard imitation learning may reproduce these inefficiencies because it treats the full demonstration as behavior to copy. The objective of this project is therefore not simply to imitate the demonstration, but to preserve its useful task intent while reducing execution inefficiency.

### 1.2 Problem Statement

This pre-lab study investigates the following question:

> Given successful-but-inefficient suboptimal demonstrations, can we learn a reward model and policy that preserve task intent while reducing execution inefficiency, eventually achieving better-than-demonstrator performance?

The demonstration is assumed to contain two kinds of information:

- **Intent information:** what the task is trying to achieve, such as moving an object to a target region.
- **Inefficiency information:** unnecessary or harmful execution patterns, such as sidetrack, high jerk, unstable contact, no-progress motion, or excessive path length.

  The goal is to learn from the demonstration without copying all of its inefficiencies.

## 2 Relation to SSRR and D-REX

SSRR/D-REX-style methods inject noise into demonstrations to generate degraded trajectories. They use the weak ordinal assumption that larger perturbations generally correspond to lower trajectory quality:

$$\tau_{\text{orig}} \succ \tau_{\text{mild}} \succ \tau_{\text{med}} \succ \tau_{\text{sev}}. \tag{1}$$

The reward model is then trained by pairwise preference fitting or ranking loss. Importantly, the method does not require hand-designed per-trajectory reward values. The degradation process itself provides weak preference supervision.

This project keeps the same ranking-based philosophy, but replaces random Gaussian noise with structured degradation. Instead of perturbing actions isotropically, the proposed method uses goal- and phase-conditioned perturbations guided by consequence features. The motivation is that isotropic Gaussian noise perturbs every action dimension equally and ignores the task phase, so many of the resulting degraded trajectories may be incoherent or uninformative about the specific inefficiencies of interest. Phase-conditioned structured perturbation instead disrupts goal-directed progress in a phase-appropriate and semantically meaningful way, which is intended to yield a cleaner and more controllable degradation signal for the reward to learn from.

Table 1: Positioning relative to SSRR/D-REX.

| Aspect | SSRR / D-REX | Proposed method |
|---|---|---|
| Degradation source | Gaussian noise magnitude | Phase-conditioned structured perturbation |
| Ranking supervision | Noise-level hierarchy | Structured degradation hierarchy |
| Inefficiency prior | Implicit in random noise | Explicit in degradation operator |
| Reward target | Feature-to-quality extrapolation | Degradation direction extrapolation |
| BfD mechanism | Reward extrapolation | Reward extrapolation |

The proposed method does not define reward as a hand-coded combination of simulator metrics, such as path length, jerk, or energy. Instead, it relocates domain prior from the reward function into the degradation operator. In other words, prior knowledge about inefficient behavior is used to define how the demonstration is perturbed, and the reward network learns from the resulting ranking. This relocation is motivated by the observation that specifying *how to make execution worse* in a given phase (for example, adding lateral drift during approach) is often easier and more robust than hand-tuning the exact weighting of a metric-based reward. The structured ranking then lets the reward network infer the relative importance of consequence features, rather than requiring those weights to be fixed manually.

# 3 Methodology Overview

The framework consists of two phases and five modules.

- **Phase 1: Suboptimal Demonstration Generation**
- **Phase 2: Demo Learning**
  - Module 1: Goal Inference and Phase Sequence Inference
  - Module 2: Consequence Forward Module and Feature Bank
  - Module 3: Phase-Conditioned Structured Degenerator
  - Module 4: Reward Network Regression
  - Module 5: Policy Improvement and BfD Search

At a high level, Phase 1 produces a demonstration that is successful but deliberately inefficient, and Phase 2 learns from it in two conceptual stages. It first extracts the task structure needed to perturb the demonstration meaningfully (the goal, the phase segmentation, and the consequence features that describe execution quality), and then it builds a structured degradation hierarchy and fits a reward to the resulting ranking. Policy improvement finally uses this reward to move away from degraded behavior. The five modules below correspond to these steps in order.

The main data flow is:

$$\tau_{\text{demo}} \to (g, z_t) \to \mathcal{B}_{\text{feat}} \to \{\tau_{\text{deg}}\} \to \mathcal{D}_{\text{rank}} \to R_\theta \to \pi^\star. \tag{2}$$

Here, $g$ is the inferred goal, $z_t$ is the phase label, $\mathcal{B}_{\text{feat}}$ is the Feature Bank, $\mathcal{D}_{\text{rank}}$ is the ranking dataset, $R_\theta$ is the learned reward, and $\pi^\star$ is the improved policy.

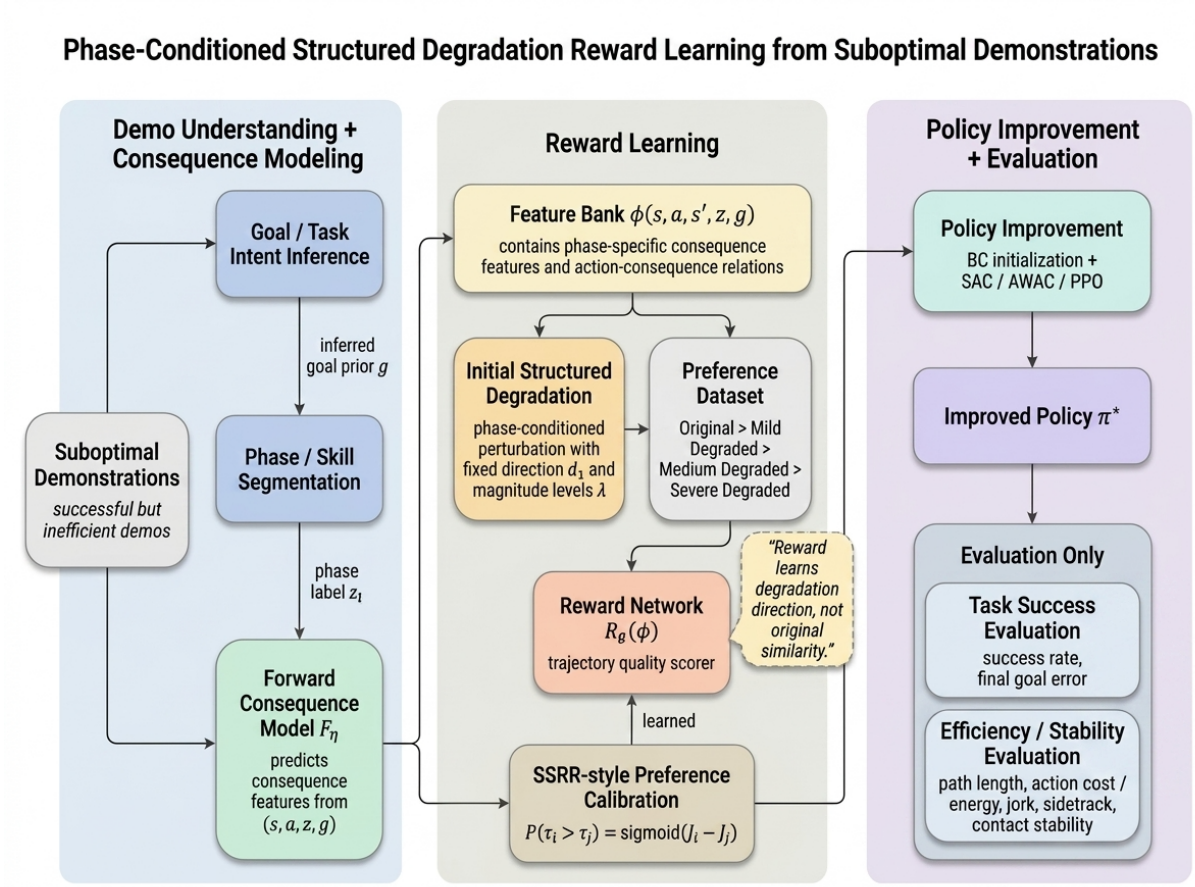


Figure 1: Overall pipeline of the proposed methodology. The main training ranking is generated from structured degradation hierarchy. Evaluation metrics are used only for final assessment, not as training ranking labels.

The important point is that the pipeline should not include an automatic reward-recalibration loop driven by simulator-efficiency validation. Such a loop would make the method look like it uses hand-designed simulator metrics to label improved trajectories. In the main proposal, improved candidates are evaluated at the end, but they are not automatically inserted into the training ranking as ground-truth labels.

## 4 Core Assumptions

This pre-lab proposal explicitly relies on the following assumptions. These are hypotheses to be tested experimentally, not claims proven in advance.

**A1. Structured degradation assumption.** Within a fixed perturbation family, increasing the perturbation level more strongly disrupts the original goal-directed behavior.

**A2. Reward extrapolation assumption.** The reward can extrapolate the degradation direction learned from "original $\succ$ degraded" data into the lower-inefficiency region beyond the original demonstration.

**A3. Feature sufficiency assumption.** The selected consequence features express the trajectory quality differences that matter. If important quality dimensions are missing, the policy may exploit the learned reward.

The better-than-demonstrator claim depends especially on A2. This report treats A2 as an empirical question: the experiments will test whether the learned reward can actually support improvement beyond the original demonstration.

# 5 Phase 1: Suboptimal Demonstration Generation

The first phase generates successful-but-inefficient demonstrations. Each demonstration should satisfy two requirements:

1. The task is completed successfully.
2. The trajectory contains visible inefficiency.

For example, a scripted policy can be modified by adding sidetrack motion, unnecessary pauses, detours, repeated alignment corrections, or unstable contact. The resulting trajectory is denoted as:

$$\tau_{\text{demo}} = \{(s_t, a_t, s_{t+1})\}_{t=0}^{T}. \tag{3}$$

This trajectory is not treated as expert data. It is treated as a successful but inefficient reference that contains both useful task intent and unnecessary execution noise.

# 6 Module 1: Goal Inference and Phase Sequence Inference

Module 1 extracts the task goal and phase sequence from the suboptimal demonstration.

## 6.1 Goal Inference

The goal $g$ represents the intended task outcome. For example:

- Pick-and-place: target object pose or target region.
- Pushing: desired object position.
- Reaching: desired end-effector pose.

In the first implementation, $g$ can be obtained from the environment-provided desired goal or task specification. Fully unsupervised goal inference is left as future work.

## 6.2 Phase Sequence Inference

The trajectory is segmented into phase labels:

$$z_t \in \{\text{Approach}, \text{Contact}, \text{Grasp}, \text{Transport}, \text{Place}, \text{Stabilize}\}. \tag{4}$$

For a pick-and-place task, the sequence may be:

$$\text{Approach} \rightarrow \text{Grasp} \rightarrow \text{Lift} \rightarrow \text{Transport} \rightarrow \text{Place}. \tag{5}$$

Phase inference is needed because the same action perturbation may have different meanings in different phases. For example, fast motion may be useful early in the approach phase but harmful near contact.

The phase inference module should be reusable for new rollouts generated during policy improvement. Otherwise, the reward network, which uses $z_t$ as an input, cannot be consistently evaluated on new trajectories.

# 7 Module 2: Consequence Forward Module and Feature Bank

The Consequence Forward Module learns how action changes affect robot motion, object interaction, and consequence features. It does not assign reward labels. Its role is to support the generation of structured degradation directions.

## 7.1 Environment-Based Consequence Learning

The environment is rolled out to collect data about action-consequence relationships. The observed feature channels may include:

- Robot-related features: gripper position, velocity, acceleration, jerk, action magnitude.
- Object-related features: object position, object velocity, object drift, object-goal distance.
- Interaction-related features: gripper-object distance, contact stability, slip or drop risk.
- Goal-related features: goal progress and final goal error.
- Trajectory-related features: path length, sidetrack tendency, no-progress tendency.

The forward model is written as:

$$F_\eta(s_t, a_t, z_t, g) \to \hat{c}_{t+1}, \tag{6}$$

where $\hat{c}_{t+1}$ is the predicted consequence feature vector.

## 7.2 Feature Bank

The phase-specific consequence information is stored in the Feature Bank:

$$\mathcal{B}_{\text{feat}} = \{B_z\}_{z \in \mathcal{Z}}. \tag{7}$$

For each phase, $B_z$ may store:

- important consequence features,
- feature correlations,
- action-consequence relations,
- goal-related features,
- demo feature distribution,
- perturbation-sensitive features.

The Feature Bank is not a reward function. It does not directly label trajectories as good or bad. It stores structured knowledge that helps generate phase-conditioned degradation directions.

### 7.3 Role of the Forward Model

To avoid making the forward model an unused module, its downstream role is explicitly defined. The forward model is used to help select degradation directions. For a given phase, it can estimate which action perturbation direction is likely to increase degradation-related consequence changes, such as sidetrack, loss of goal progress, unstable contact, or object drift.

However, two constraints are required.

**Fixed direction per degradation family.** Each degradation family uses a fixed direction $d_z$ and varying magnitude $\lambda$:

$$a'_t = a_t + \lambda d_z. \tag{8}$$

The forward model may help choose $d_z$, but once a family is defined, $d_z$ is held fixed across perturbation levels. This preserves the nested ranking structure.

**Actual rollout features for reward learning.** The forward model predictions are used to select degradation directions, not to provide reward labels. The reward network is trained using consequence features extracted from actual executed rollouts, not only predicted features. This prevents forward-model error from leaking directly into the learned reward.

## 8 Module 3: Phase-Conditioned Structured Degenerator

Module 3 generates degraded trajectories from the original suboptimal demonstration. It replaces the Gaussian noise injection used in SSRR/D-REX with phase-conditioned structured perturbations.

### 8.1 Structured Perturbation

For each phase, perturbations are designed to disrupt the original goal-directed behavior. The guiding principle is that each perturbation should make the execution less efficient in a way that is characteristic of that phase, rather than adding generic noise: it should push the relevant consequence features (for example, lateral deviation in approach, or object drift in transport) in the direction that the demonstration was trying to avoid.

Examples include:

- Approach phase: lateral deviation, unnecessary sidetrack, delayed approach.
- Contact/Grasp phase: misalignment, early or late contact, unstable grasp.
- Transport/Push phase: object drift, unstable contact, reduced goal progress.
- Place/Release phase: unstable release, poor settling, release timing perturbation.

Increasing the magnitude $\lambda$ in Equation (8) creates a degradation hierarchy:

$$\tau_{\text{orig}} \succ \tau_{\text{mild}} \succ \tau_{\text{med}} \succ \tau_{\text{sev}} \succ \tau_{\text{super}}. \tag{9}$$

This ranking is a weak ordinal assumption, analogous to the noise-level assumption in SSRR/D-REX.

### 8.2 Nested Perturbation Pairs

Because phase-conditioned perturbations can be vector-valued, not all degraded trajectories are safely comparable. For example, one trajectory may have mild approach perturbation but severe transport perturbation, while another may have severe approach perturbation but mild transport perturbation.

To avoid inconsistent ranking labels, preference pairs are formed only when the perturbation level increases within the same degradation family:

$$\tau_{\text{orig}} \succ \tau_{\lambda_1 d_z} \succ \tau_{\lambda_2 d_z} \succ \tau_{\lambda_3 d_z}, \qquad 0 < \lambda_1 < \lambda_2 < \lambda_3. \tag{10}$$

Cross-profile comparisons are excluded unless additional preference information is available. This keeps the ranking logically consistent even when perturbations are phase-conditioned.

## 9 Module 4: Reward Network Regression

Module 4 trains a reward network using the ranking dataset generated by Module 3.

### 9.1 Reward Input and Return

Each timestep is represented by a phase-conditioned consequence feature:

$$\phi_t = \phi(s_t, a_t, s_{t+1}, z_t, g). \tag{11}$$

The reward network outputs a scalar reward:

$$r_t = R_\theta(\phi_t). \tag{12}$$

The trajectory return is:

$$J_\theta(\tau) = \sum_{t=0}^{T} R_\theta(\phi_t). \tag{13}$$

### 9.2 Pairwise Preference Fitting

For a preference pair $\tau_i \succ \tau_j$, the model predicts:

$$P(\tau_i \succ \tau_j) = \sigma\left(J_\theta(\tau_i) - J_\theta(\tau_j)\right), \tag{14}$$

where $\sigma(\cdot)$ is the sigmoid function.

The ranking loss is:

$$\mathcal{L}_{\text{rank}} = -\sum_{(\tau_i, \tau_j) \in \mathcal{D}_{\text{rank}}} \log P(\tau_i \succ \tau_j). \tag{15}$$

### 9.3 Learning Degradation Direction, Not Original Similarity

The reward network should not simply learn similarity to the original demonstration. If it only assigns high reward to trajectories near the original, then policy improvement may collapse back to imitation.

Instead, the reward network should learn the latent degradation direction induced by the structured perturbation hierarchy. In other words, it should learn which consequence-feature changes are associated with lower quality as perturbation magnitude increases.

This makes it possible for policy improvement to search for trajectories that are not merely similar to the original, but have lower predicted degradation than the original demonstration.

### 9.4 BfD Extrapolation Assumption

The training ranking contains only:

$$\text{Original} \succ \text{Degraded}. \tag{16}$$

Therefore, trajectories better than the original demonstration are not directly observed during reward training.

Better-than-demonstrator learning therefore depends on the reward extrapolation assumption: the learned reward can extrapolate the degradation direction beyond the original demonstration into a lower-inefficiency region. This is not guaranteed. It is an empirical hypothesis to be tested in simulation.

The intuition for why this might succeed follows the T-REX/D-REX precedent. Because the reward is a function of consequence features rather than of trajectory identity, it can be evaluated on feature values that were never seen during training. If the learned reward captures an approximately monotonic relationship between inefficiency-related features and trajectory quality, then trajectories with lower inefficiency than the demonstration receive higher reward, and policy improvement is pulled toward them. Since all training data lie on the degraded side of the demonstration, whether the reward keeps increasing in the unobserved lower-inefficiency region is exactly what the experiments must establish.

### 9.5 Ordinal Consistency

Within each degradation family, the reward should satisfy:

$$\lambda_1 < \lambda_2 \quad \Rightarrow \quad J_\theta(\tau_{\lambda_1 d_z}) > J_\theta(\tau_{\lambda_2 d_z}). \tag{17}$$

This is a necessary condition for learning a stable degradation direction, but it does not by itself guarantee BfD. It constrains the reward on the observed degraded side. If early experiments show the reward becoming flat near the original demonstration, a monotonic reward architecture can be considered as a future modification.

## 10 Module 5: Policy Improvement and BfD Search

Given the learned reward $R_\theta$, the policy is improved by maximizing the expected learned return:

$$\pi^\star = \arg\max_\pi \mathbb{E}_{\tau\sim\pi}\left[J_\theta(\tau)\right]. \tag{18}$$

Possible optimization methods include:

- SAC,
- AWAC,
- PPO,
- IAC-style actor-critic,
- trajectory optimization.

A practical first implementation is behavioral cloning initialization followed by reward-based policy improvement. Behavioral cloning initialization keeps the policy within the feasible region around the demonstration at the start of training, which stabilizes the subsequent reward-based

optimization; the learned reward then drives the policy away from degraded feature patterns toward lower-inefficiency behavior.

The goal of Module 5 is not to reproduce the original demonstration. The goal is to avoid learned degradation patterns and search for trajectories with lower predicted inefficiency. The resulting policy is then evaluated against the original demonstrator.

# 11 Algorithmic Summary

Table 2: Algorithmic summary of the proposed methodology.

| Step | Operation |
|---|---|
| 1 | Generate or load a successful-but-inefficient demonstration $\tau_{\text{demo}}$. |
| 2 | Infer the goal $g$ and phase labels $z_t$. |
| 3 | Collect environment rollouts and train the consequence forward model $F_\eta$. |
| 4 | Build the phase-conditioned Feature Bank $\mathcal{B}_{\text{feat}}$. |
| 5 | For each phase, select a fixed degradation direction $d_z$ using $\mathcal{B}_{\text{feat}}$ and $F_\eta$. |
| 6 | Generate degraded trajectories using $a'_t = a_t + \lambda d_z$ for several levels $\lambda$. |
| 7 | Execute or replay trajectories and extract actual consequence features. |
| 8 | Construct nested ranking pairs and train $R_\theta$ by pairwise ranking loss. |
| 9 | Initialize a policy by behavioral cloning and improve it using the learned reward. |
| 10 | Evaluate the improved policy against the original demonstrator. |

# 12 Experimental Plan

## 12.1 Environment

The initial experiment will use simulated manipulation environments such as:

- MuJoCo Fetch Pick-and-Place,
- Push-and-Slide,
- Reach,
- simple object transport tasks.

Suboptimal demonstrations will be created by injecting structured inefficiencies into scripted successful policies.

## 12.2 Baselines

Table 3: Planned comparison methods.

| Method | Description |
|---|---|
| Behavioral Cloning | Directly imitates the suboptimal demonstration |
| D-REX / SSRR | Uses random-noise degradation ranking |
| Ours without phase conditioning | Uses structured degradation without phase labels |
| Ours without forward model | Uses hand-defined perturbation directions only |
| Ours full | Uses phase-conditioned structured degradation with Feature Bank |

### 12.3 Evaluation Metrics

Simulator metrics are used only for evaluation, not as training ranking labels. Metrics include:

- task success rate,
- final goal error,
- path length,
- execution time,
- cumulative action cost or energy proxy,
- jerk,
- object drift,
- contact stability,
- no-progress ratio,
- sidetrack score.

A policy is considered better-than-demonstrator if it maintains task success while improving at least one or more efficiency metrics relative to the original demonstrator.

An important test is whether the policy improves metrics that were not directly targeted by the structured Degenerator. This can provide evidence that the reward learned a generalizable degradation direction rather than merely reversing the injected perturbation.

### 12.4 Ablation Studies

Possible ablations include:

- removing phase conditioning,
- removing the forward model,
- using Gaussian perturbation instead of structured perturbation,
- using non-nested ranking pairs,
- comparing actual rollout features versus predicted features,
- varying the consequence feature set.

## 13 Assumptions and Scope

The method avoids hand-designed reward functions, but it still encodes inefficiency priors in the structured degradation operator. This is an intentional design choice. The experiments should verify that the learned policy does more than simply undo the operator.

The method also does not claim global optimality. The goal is local improvement around the demonstration distribution. Specifically, the method aims to reduce local inefficiencies such as sidetrack, high jerk, no-progress motion, or unstable contact while preserving task success.

## 14 Expected Contributions

The expected contributions of this pre-lab study are:

1. A better-than-demonstrator learning framework for successful-but-inefficient manipulation demonstrations.
2. An extension of SSRR/D-REX-style degradation ranking from Gaussian noise to phase-conditioned structured perturbation.
3. A Consequence Forward Module and Feature Bank for supporting targeted degradation generation.
4. A reward learning formulation that learns degradation direction rather than original similarity.
5. An empirical study of whether structured degradation ranking can support local better-than-demonstrator policy improvement.

## 15 Limitations and Future Work

Several limitations remain:

- **Reward extrapolation may fail.** The reward may become flat near the original demonstration or fail to generalize to lower-inefficiency trajectories.
- **Feature basis may be incomplete.** If important task quality dimensions are missing, the policy may exploit the learned reward.
- **Forward-model error may weaken degradation.** If the forward model selects poor perturbation directions, the degradation hierarchy may be weak.
- **Structured degradation encodes prior knowledge.** The method avoids hand-designed reward functions, but prior knowledge still enters through the degradation operator.
- **Global optimality is not guaranteed.** The method aims for local improvement, not discovery of a globally optimal strategy.

Future work may include monotonic reward architectures, human-preference-based recalibration, automatic feature discovery, and multi-task manipulation settings.

## 16 Conclusion

This report proposed a pre-lab methodology for learning better-than-demonstrator manipulation policies from successful-but-inefficient demonstrations. The method follows the ranking-based reward learning philosophy of SSRR/D-REX, but replaces random Gaussian degradation with phase-conditioned structured degradation.

The core idea is to generate a weak ordinal hierarchy of degraded trajectories using structured action perturbations. A reward network is then trained using pairwise preference fitting over phase-conditioned consequence features. Unlike imitation learning, the reward model should not learn simple similarity to the original demonstration. Instead, it should learn the latent degradation direction induced by the structured perturbation hierarchy.

Policy improvement then maximizes the learned reward to search for trajectories with lower predicted inefficiency than the original demonstration. Since better-than-demonstrator behavior

depends on reward extrapolation beyond the original demonstration, this framework treats the extrapolation assumption as an empirical hypothesis to be tested in simulation.